

Tugas 10: Tugas Praktikum Mandiri 10

Revani – 0110224111¹

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224111@student.nurulfikri.ac.id

Abstract. Pada tugas latihan praktikum 10 ini yang terdiri dari Latihan 1, 2 dan 3. Dilakukan serangkaian percobaan untuk memahami konsep algoritma K-Nearest Neighbors (KNN) dan evaluasi performa model menggunakan Confusion Matrix. Praktikum terdiri atas tiga studi kasus yaitu prediksi persepsi kenyamanan cuaca oleh seseorang berdasarkan suhu dan kecepatan angin, evaluasi prediksi kelulusan mahasiswa berdasarkan hasil sebenarnya dan prediksi sistem, serta prediksi tipe cuaca dari berbagai parameter atmosfer seperti temperatur, kelembaban, kecepatan angin, tekanan udara, dan visibilitas. Tujuan utama dari kegiatan ini adalah untuk memberikan pemahaman praktis mengenai proses preprocessing data, pemodelan KNN, serta pengukuran performanya menggunakan metrik seperti akurasi, precision, recall, dan visualisasi confusion matrix.

Hasil percobaan menunjukkan bahwa KNN dapat digunakan untuk melakukan klasifikasi pada berbagai jenis data dalam kehidupan nyata, tetapi kualitas prediksi sangat bergantung pada pemilihan parameter seperti nilai K serta proses pengolahan data seperti normalisasi dan encoding. Pada studi kasus prediksi persepsi cuaca dan prediksi kondisi cuaca, KNN memberikan hasil akurasi yang baik setelah dilakukan pemilihan nilai K terbaik dan scaling pada fitur. Sedangkan pada evaluasi prediksi kelulusan mahasiswa, terlihat bahwa meskipun model dapat memberikan akurasi yang cukup baik, masih terdapat kesalahan klasifikasi sehingga performanya belum optimal. Secara keseluruhan, praktikum ini menunjukkan bahwa KNN merupakan algoritma sederhana namun efektif dalam tugas klasifikasi ketika diproses dengan langkah analisis yang benar.

LATIHAN 1

1. Penjelasan Hasil Latihan 1

Pada latihan ini menggunakan algoritma *K-Nearest Neighbors (KNN)* untuk memprediksi persepsi seseorang terhadap suhu udara berdasarkan dua informasi, yaitu temperatur udara dan kecepatan angin. Dataset diambil dari Google Sheets yang kemudian diproses menggunakan Python serta library *scikit-learn*. Setelah model dilatih menggunakan sebagian data, model melakukan prediksi terhadap sebuah data baru yaitu kondisi cuaca (16°C dan kecepatan angin 3 km/jam). Hasil prediksi menunjukkan bahwa pada kondisi tersebut, persepsi yang muncul adalah dingin. Selain itu dilakukan juga pencarian nilai k terbaik menggunakan *elbow-test*, serta validasi hasil prediksi dengan perhitungan manual sesuai tabel Excel yang diberikan. Hasil yang diperoleh konsisten sehingga model dianggap bekerja sesuai teori. Berikut ini adalah hasil praktikum yang saya kerjakan pada praktikum 10 latihan 1.

2. Penjelasan Kode Program

Dataset yang digunakan seperti yang ditunjukkan pada Gambar 1.

Data Training					
Suhu	Angin	Kelas	suhu	angin	jarak
10	0	Dingin	36	9	6.708203932
25	0	Panas	81	9	9.486832981
15	5	Dingin	1	4	2.236067977
20	3	Panas	16	0	4
18	7	Dingin	4	16	4.472135955
20	10	Dingin	16	49	8.062257748
22	5	Panas	36	4	6.32455532
24	3	Panas	64	0	8
Data Testing	k	5			
Suhu	Angin	Kelas			
16	3	dingin			

Gambar 1. Pada bagian ini ditunjukkan dataset.

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

Library ini digunakan untuk membaca data, membangun model KNN, membagi data train/test, dan menghitung akurasi seperti yang ditunjukkan pada Gambar 2.

```
Import Library

import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

Gambar 2. Pada bagian ini ditunjukkan import library.

```
df =
pd.read_excel("https://docs.google.com/uc?export=download&id=1aBIMc_eKJQIXdbOP-ccAC2W0knzK_RWY")
df
```

Kode membaca dataset cuaca berisi suhu, angin, dan persepsi Dingin/Panas. Output yang dihasilkan yaitu 5 baris pertama seperti yang ditunjukkan pada Gambar 3.

```
df = pd.read_excel("https://docs.google.com/uc?export=download&id=1aBIMc_eKJQIXdbOP-ccAC2W0knzK_RWY")
df
```

	temperatur_udara	kecepatan_angin	klasifikasi_persepsi	marry
0	10	0	dingin	
1	25	0	panas	
2	15	5	dingin	
3	20	3	panas	
4	18	7	dingin	
5	20	10	dingin	
6	22	5	panas	
7	24	6	panas	

Gambar 3. Pada bagian ini ditunjukkan untuk membaca dataset.

```
df.info()
```

Menampilkan jumlah data, tipe data, dan apakah ada missing value seperti yang ditunjukkan pada Gambar 4.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 3 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   temperatur_udara                      8 non-null      int64
1   kecepatan_angin                      8 non-null      int64
2   klasifikasi_persepsi marry           8 non-null      object
dtypes: int64(2), object(1)
memory usage: 324.0+ bytes
```

Gambar 4. Pada bagian ini ditunjukkan informasi dataset.

```
X = df[['temperatur_udara', 'kecepatan_angin']]
y = df['klasifikasi_persepsi marry']
```

Menentukan fitur dan label dengan x berisi fitur temperatur dan kecepatan angin, serta y berisi label dingin/panas seperti yang ditunjukkan pada Gambar 5.

Fitur & Label

```
X = df[['temperatur_udara', 'kecepatan_angin']]
y = df['klasifikasi_persepsi marry']
```

Gambar 5. Pada bagian ini ditunjukkan fitur dan label.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=42
)
X_train, X_test
```

Membagi data ke train dan test dengan 60% untuk training dan 40% untuk testing seperti yang ditunjukkan pada Gambar 6.

Split Data Train - Test

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.4, random_state=42  
)
```

X_train, X_test

	temperatur_udara	kecepatan_angin
2	15	5
4	18	7
3	20	3
6	22	5,
	temperatur_udara	kecepatan_angin
1	25	0
5	20	10
0	10	0
7	24	6)

Gambar 6. Pada bagian ini ditunjukkan split data train dan test.

```
model = KNeighborsClassifier(n_neighbors=3)  
model.fit(X_train, y_train)
```

Melatih model dengan 3 tetangga terdekat.

```
y_pred = model.predict(X_test)  
acc = accuracy_score(y_test, y_pred)  
print("Akurasi model (k):", acc)
```

Menghitung akurasi seperti yang ditunjukkan pada Gambar 7.

Training Model KNN

```
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
```

▼ KNeighborsClassifier ⓘ ?

KNeighborsClassifier(n_neighbors=3)

```
y_pred = model.predict(X_test)
acc = accuracy_score(y_test, y_pred)
```

```
print("Akurasi model (k):", acc)
```

Akurasi model (k): 0.75

Gambar 7. Pada bagian ini ditunjukkan training model KNN.

```
test_data = pd.DataFrame([[16, 3]], columns=X.columns)
hasil = model.predict(test_data)

print("Persepsi Marry (16°C, 3 km/jam):", hasil[0])
```

Mengecek kondisi baru yaitu suhu 16°C dan angin 3 km/jam. Model menentukan apakah manusia merasa panas atau dingin. Output yang dihasilkan yaitu model memprediksi dingin seperti yang ditunjukkan pada Gambar 8.

Prediksi Data Test

```
test_data = pd.DataFrame([[16, 3]], columns=X.columns)
hasil = model.predict(test_data)

print("Persepsi Marry (16°C, 3 km/jam):", hasil[0])
```

Persepsi Marry (16°C, 3 km/jam): dingin

Gambar 8. Pada bagian ini ditunjukkan prediksi.

```
akurasi_k = {}

for k in range(1, len(X_train) + 1):
    model = KNeighborsClassifier(n_neighbors=k)
```

```
model.fit(X_train, y_train)
pred = model.predict(X_test)
akurasi_k[k] = accuracy_score(y_test, pred)
```

akurasi_k

Melakukan perhitungan akurasi untuk setiap nilai k, dengan output k=1 menghasilkan 100% terbaik seperti yang ditunjukkan pada Gambar 9.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average='macro')
rec = recall_score(y_test, y_pred, average='macro')

print("Akurasi :", round(acc*100, 2), "%")
print("Precision :", round(prec*100, 2), "%")
print("Recall :", round(rec*100, 2), "%")

Akurasi : 89.17 %
Precision : 89.22 %
Recall : 89.08 %
```

Gambar 9. Pada bagian ini ditunjukkan nilai k terbaik.

```
k_terbaik = max(akurasi_k, key=akurasi_k.get)
print("Nilai k terbaik:", k_terbaik)
print("Akurasi:", akurasi_k[k_terbaik])
```

Untuk menampilkan nilai k terbaik dengan output k=1 yang paling akurat untuk dataset ini seperti yang ditunjukkan pada Gambar 10.

Menampilkan K Terbaik

```
k_terbaik = max(akurasi_k, key=akurasi_k.get)
print("Nilai k terbaik:", k_terbaik)
print("Akurasi:", akurasi_k[k_terbaik])

Nilai k terbaik: 1
Akurasi: 1.0
```

Gambar 10. Pada bagian ini menampilkan nilai k terbaik 1.

3. Kesimpulan dan Hasil Implementasi

Dari hasil implementasi model dapat memprediksi persepsi manusia terhadap cuaca berdasarkan kondisi lingkungan. Ini dapat diterapkan dalam aplikasi cuaca atau IoT untuk

menentukan kenyamanan pengguna secara otomatis, misalnya pengaturan AC otomatis berdasarkan situasi suhu dan angin.

Dari praktikum ini dapat disimpulkan bahwa model KNN mampu melakukan prediksi terhadap persepsi suhu dengan akurasi yang cukup baik. Hasil perhitungan manual menggunakan Excel juga membuktikan bahwa model berjalan sesuai teori *Nearest Neighbor*. Pemilihan nilai k sangat memengaruhi akurasi dan untuk dataset ini nilai terbaik adalah $k = 1$. Secara keseluruhan, model mampu mengklasifikasikan bahwa kondisi 16°C dengan angin 3 km/jam menghasilkan persepsi dingin.

LATIHAN 2

1. Penjelasan Hasil Latihan 2

Pada latihan ini dataset berisi 10 data mahasiswa dengan informasi hasil sebenarnya dan hasil prediksi kelulusan. Tujuan praktikum adalah mengevaluasi seberapa tepat model dalam memprediksi apakah mahasiswa lulus atau tidak. Evaluasi dilakukan menggunakan Confusion Matrix, Accuracy, Precision, dan Recall. Dari hasil yang diperoleh terlihat bahwa model cukup baik dalam memprediksi kelulusan, tetapi masih terdapat kesalahan klasifikasi pada beberapa data sehingga performanya belum sempurna. Berikut ini adalah hasil praktikum yang saya kerjakan pada praktikum 10 latihan 2.

2. Penjelasan Kode Program

Dataset yang digunakan seperti yang ditunjukkan pada Gambar 11.

nim	hasil_sebenarnya	hasil_prediksi
TI001	Lulus	Lulus
TI002	Lulus	Lulus
TI003	Lulus	Lulus
TI004	Lulus	Tidak Lulus
TI005	Lulus	Tidak Lulus
TI006	Tidak Lulus	Lulus
TI007	Tidak Lulus	Tidak Lulus
TI008	Tidak Lulus	Tidak Lulus
TI009	Tidak Lulus	Tidak Lulus
TI010	Tidak Lulus	Tidak Lulus

Gambar 11. Pada bagian ini ditunjukkan dataset.

```
import pandas as pd
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score
import matplotlib.pyplot as plt
```

Bagian pertama dari program yaitu melakukan import library. Library pandas digunakan untuk membaca dataset, sedangkan confusion_matrix, accuracy_score, precision_score, dan recall_score digunakan untuk menghitung performa evaluasi. Selain itu, matplotlib.pyplot digunakan untuk menampilkan visualisasi confusion matrix. Kode ini tidak menghasilkan output visual namun merupakan persiapan fungsi yang akan digunakan seperti yang ditunjukkan pada Gambar 12.

Import Library

```
import pandas as pd
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
import matplotlib.pyplot as plt
```

Gambar 12. Pada bagian ini ditunjukkan import library.

```
df1 =
pd.read_excel("https://docs.google.com/uc?export=download&id=1r5b5XXM7h9
qdg8JM1ZQtRld0kgQgy3ZQ")
df1
```

Selanjutnya dataset diambil dari Google Drive menggunakan format Excel dan ditampilkan dalam bentuk tabel. Data berisi tiga kolom yaitu NIM, hasil_sebenarnya, dan hasil_prediksi. Output berupa tabel berisi mahasiswa yang lulus atau tidak lulus serta hasil prediksi dari model seperti yang ditunjukkan pada Gambar 13.

```
df1 = pd.read_excel("https://docs.google.com/uc?export=download&id=1r5b5XXM7h9qdg8JM1ZQtRld0kgQgy3ZQ")
df1
```

	nim	hasil_sebenarnya	hasil_prediksi
0	TI001	Lulus	Lulus
1	TI002	Lulus	Lulus
2	TI003	Lulus	Lulus
3	TI004	Lulus	Tidak Lulus
4	TI005	Lulus	Tidak Lulus
5	TI006	Tidak Lulus	Lulus
6	TI007	Tidak Lulus	Tidak Lulus
7	TI008	Tidak Lulus	Tidak Lulus
8	TI009	Tidak Lulus	Tidak Lulus
9	TI010	Tidak Lulus	Tidak Lulus

Gambar 13. Pada bagian ini ditunjukkan untuk membaca dataset.

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
df1["Actual_enc"] = encoder.fit_transform(df1["hasil_sebenarnya"])
df1["Pred_enc"] = encoder.transform(df1["hasil_prediksi"])

y_true = df1["Actual_enc"]
y_pred = df1["Pred_enc"]
```

Karena machine learning lebih mudah memproses angka daripada teks, maka dilakukan encoding pada kolom hasil sebenarnya dan hasil prediksi. LabelEncoder digunakan untuk mengubah kata “Lulus” menjadi angka 0 dan “Tidak Lulus” menjadi angka 1. Setelah perintah ini dieksekusi, muncul dua kolom baru yaitu Actual_enc dan Pred_enc yang berisi angka hasil encoding. Kemudian nilai yang sudah diencode dimasukkan ke variabel y_true sebagai nilai sebenarnya dan y_pred sebagai nilai prediksi. Kedua variabel ini dipakai untuk perhitungan confusion matrix dan metrik evaluasi. Perintah ini tidak menampilkan output namun menyiapkan data untuk evaluasi seperti yang ditunjukkan pada Gambar 14.

```

from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
df1["Actual_enc"] = encoder.fit_transform(df1["hasil_sebenarnya"])
df1["Pred_enc"] = encoder.transform(df1["hasil_prediksi"])

y_true = df1["Actual_enc"]
y_pred = df1["Pred_enc"]

```

Gambar 14. Pada bagian ini ditunjukkan encoding.

```

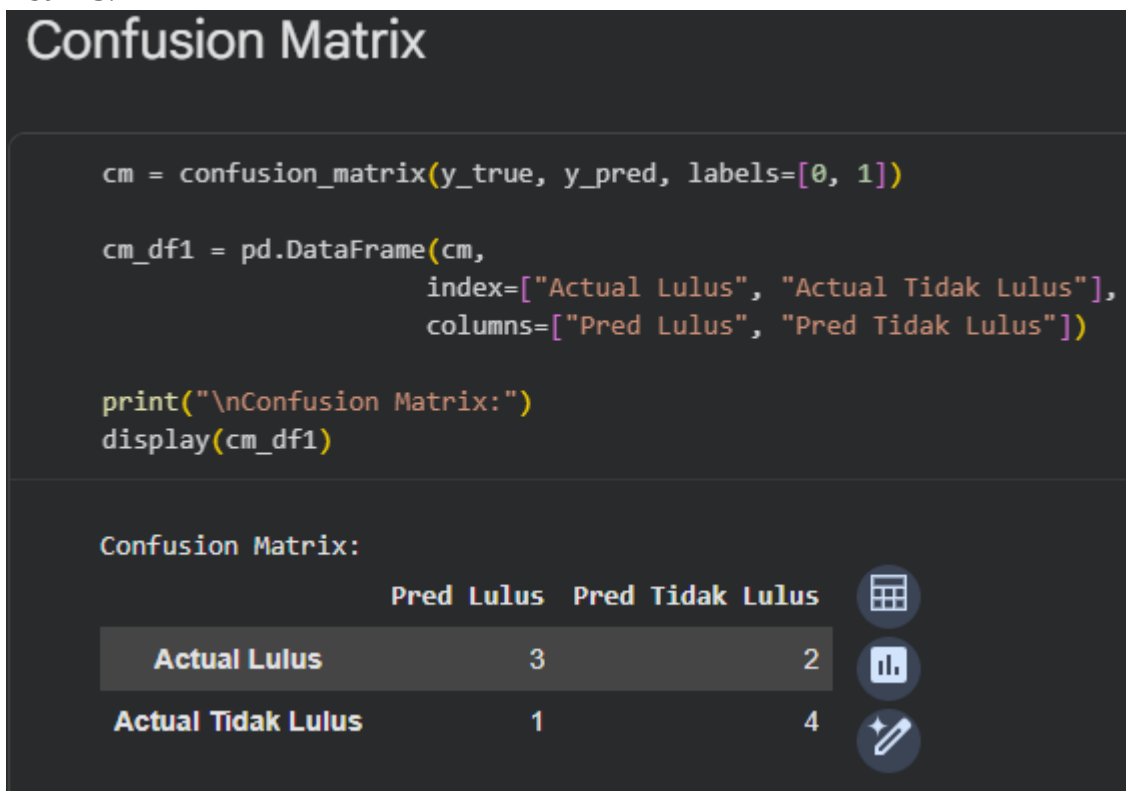
cm = confusion_matrix(y_true, y_pred, labels=[0, 1])

cm_df1 = pd.DataFrame(cm,
                       index=["Actual Lulus", "Actual Tidak Lulus"],
                       columns=["Pred Lulus", "Pred Tidak Lulus"])

print("\nConfusion Matrix:")
display(cm_df1)

```

Confusion Matrix dihitung agar dapat mengetahui berapa banyak prediksi yang benar dan salah. Outputnya berupa tabel 2x2 yang menunjukkan jumlah data benar dan salah diklasifikasikan. Format matriks disajikan lebih informatif dengan label Actual Lulus dan Actual Tidak Lulus pada baris serta Pred Lulus dan Pred Tidak Lulus pada kolom seperti yang ditunjukkan pada Gambar 15.



Gambar 15. Pada bagian ini ditunjukkan confusion matrix.

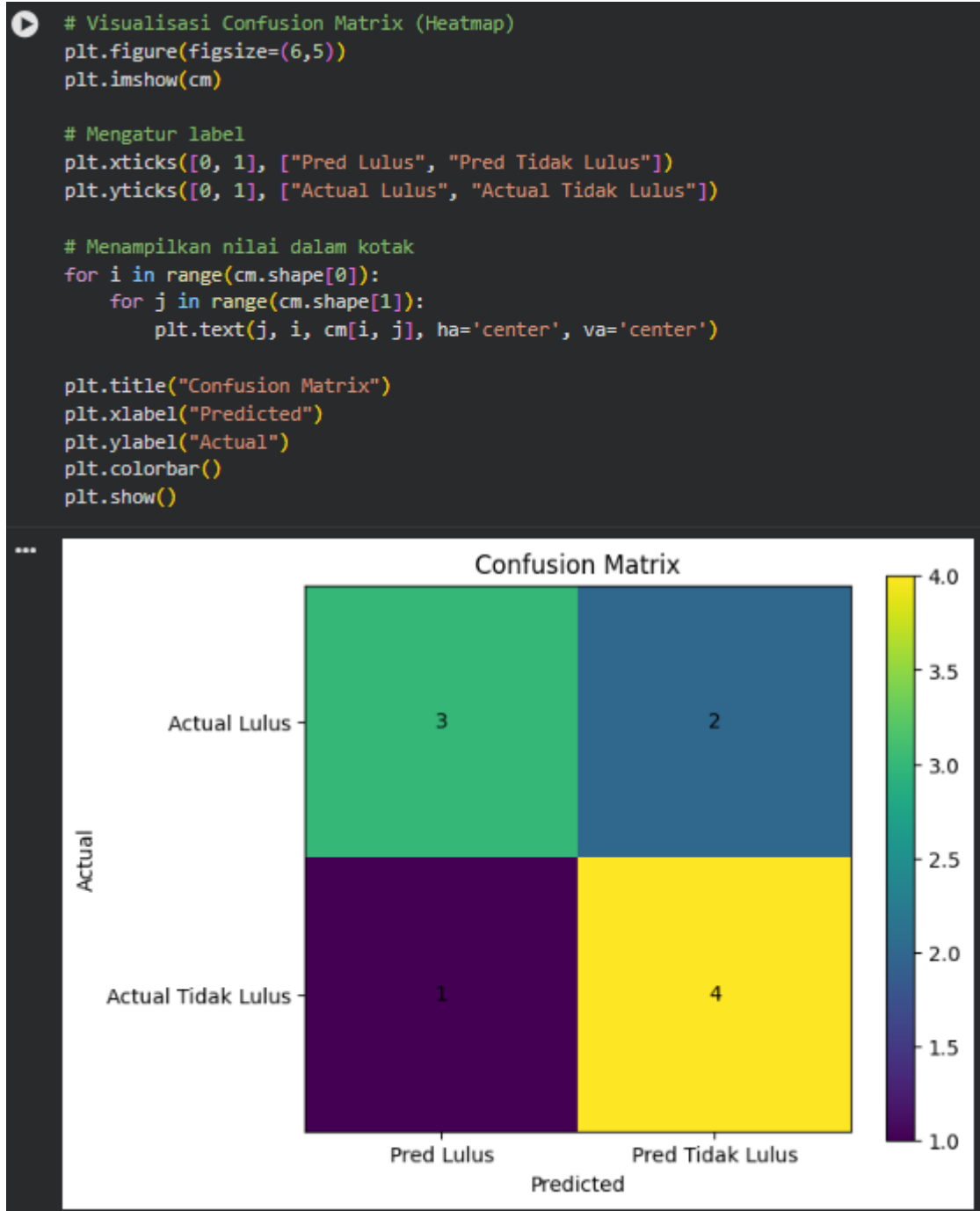
```
# Visualisasi Confusion Matrix (Heatmap)
plt.figure(figsize=(6,5))
plt.imshow(cm)

# Mengatur label
plt.xticks([0, 1], ["Pred Lulus", "Pred Tidak Lulus"])
plt.yticks([0, 1], ["Actual Lulus", "Actual Tidak Lulus"])

# Menampilkan nilai dalam kotak
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, cm[i, j], ha='center', va='center')

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()
plt.show()
```

Confusion Matrix tersebut kemudian divisualisasikan dalam bentuk heatmap. Setiap angka hasil perhitungan matrix ditampilkan pada kotak yang sesuai sehingga lebih mudah dibaca. Warna menunjukkan intensitas jumlah data dan semakin gelap warnanya menandakan jumlah data pada kategori tersebut semakin banyak seperti yang ditunjukkan pada Gambar 16.



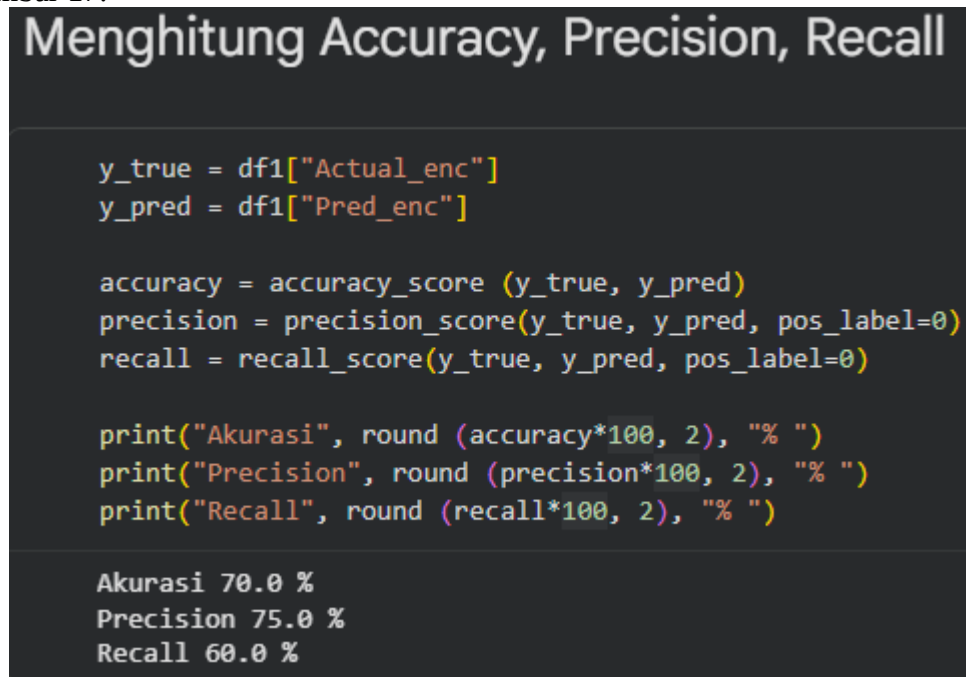
Gambar 16. Pada bagian ini ditunjukkan visualisasi confusion matrix.

```
y_true = df1["Actual_enc"]
y_pred = df1["Pred_enc"]

accuracy = accuracy_score(y_true, y_pred)
precision = precision_score(y_true, y_pred, pos_label=0)
recall = recall_score(y_true, y_pred, pos_label=0)
```

```
print("Akurasi", round (accuracy*100, 2), "% ")
print("Precision", round (precision*100, 2), "% ")
print("Recall", round (recall*100, 2), "% ")
```

Tahap terakhir yaitu menghitung nilai Accuracy, Precision, dan Recall. Accuracy menunjukkan tingkat ketepatan model secara keseluruhan, precision menunjukkan seberapa akurat model memprediksi kelas Lulus, dan recall menunjukkan seberapa banyak mahasiswa yang seharusnya lulus diprediksi dengan benar. Output yang dihasilkan seperti yang ditunjukkan pada Gambar 17.



```
Menghitung Accuracy, Precision, Recall

y_true = df1["Actual_enc"]
y_pred = df1["Pred_enc"]

accuracy = accuracy_score (y_true, y_pred)
precision = precision_score(y_true, y_pred, pos_label=0)
recall = recall_score(y_true, y_pred, pos_label=0)

print("Akurasi", round (accuracy*100, 2), "% ")
print("Precision", round (precision*100, 2), "% ")
print("Recall", round (recall*100, 2), "% ")

Akurasi 70.0 %
Precision 75.0 %
Recall 60.0 %
```

Gambar 17. Pada bagian ini menghitung accuracy, precision, recall.

3. Kesimpulan dan Hasil Implementasi

Dari hasil implementasi dengan nilai Akurasi sebesar 70%, dapat dikatakan bahwa model sudah cukup baik dalam memprediksi hasil kelulusan mahasiswa namun belum sempurna. Precision dan Recall masih menunjukkan adanya kesalahan pada prediksi mahasiswa yang seharusnya lulus namun diprediksi tidak lulus. Artinya model masih perlu perbaikan, misalnya dengan menambah data latih atau algoritma yang lebih baik.

Dari praktikum ini dapat disimpulkan bahwa model prediksi kelulusan yang digunakan mampu memberikan ketepatan klasifikasi sebesar 70%. Namun terdapat beberapa mahasiswa yang salah diklasifikasikan, sehingga performa model belum maksimal. Confusion Matrix memberikan gambaran yang jelas terkait pola kesalahan prediksi dan dapat menjadi dasar untuk peningkatan model. Dengan demikian, metode evaluasi seperti ini sangat penting untuk memastikan sistem prediksi berjalan dengan baik sebelum diterapkan secara nyata.

LATIHAN 3

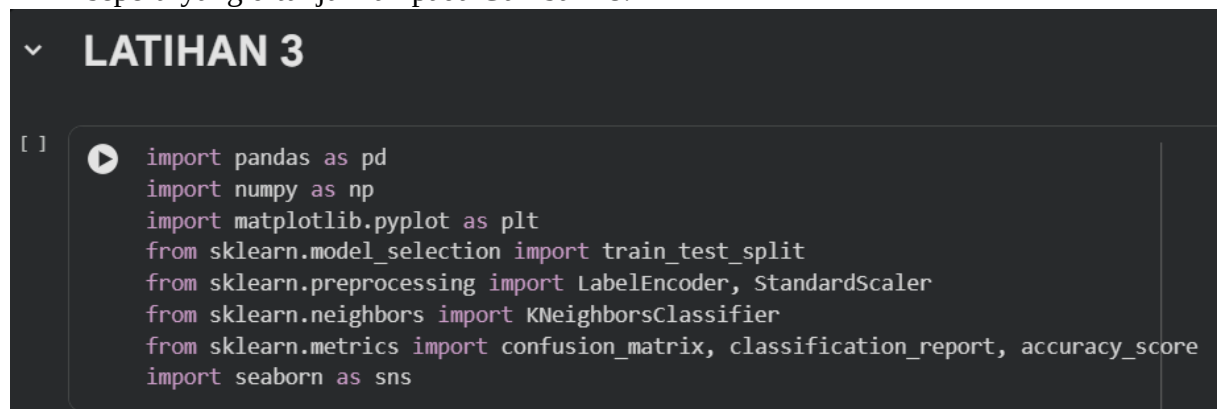
1. Penjelasan Hasil Latihan 3

Pada latihan ini saya melakukan prediksi jenis cuaca menggunakan algoritma klasifikasi K-Nearest Neighbors (KNN). Dataset yang digunakan berisi informasi berbagai kondisi cuaca seperti temperatur, kelembaban, kecepatan angin, curah hujan, tekanan udara, indeks UV, musim dan lokasi. Target prediksinya adalah kolom “Weather Type” yang berisi label cuaca seperti Rainy, Cloudy, Sunny, Snowy, dan sebagainya. Proses analisis dimulai dari pemanggilan dataset, pengecekan tipe data serta nilai kosong, melakukan encoding terhadap data kategori agar bisa diolah model, kemudian melakukan pembagian data menjadi training dan testing. Data numerik kemudian ditransformasi menggunakan StandarScaler agar perhitungan jarak pada algoritma KNN menjadi lebih akurat. Model KNN dilatih menggunakan data training dan dilakukan pengujian dengan data testing untuk menghitung akurasi prediksinya. Hasil evaluasi menunjukkan nilai akurasi, precision, dan recall yang menjelaskan seberapa baik model dalam memprediksi cuaca. Confusion Matrix juga ditampilkan untuk melihat keberhasilan prediksi berdasarkan masing-masing kategori cuaca. Berikut ini adalah hasil praktikum yang saya kerjakan pada praktikum 10 latihan 3.

2. Penjelasan Kode Program

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report,
accuracy_score
import seaborn as sns
```

Pada bagian awal program dilakukan proses pemanggilan library seperti pandas, numpy, matplotlib, seaborn, serta beberapa fungsi dari scikit-learn yang diperlukan dalam pemodelan KNN seperti yang ditunjukkan pada Gambar 18.



Gambar 18. Pada bagian ini ditunjukkan import library.

```
import pandas as pd
df2 = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/Machine Learning/Praktikum10/Data/weather_classification_data.csv")
df2.head()
```

Setelah itu, dataset cuaca di-load menggunakan perintah `pd.read_csv()`. Kemudian `df2.head()` digunakan untuk melihat contoh lima baris data teratas agar memastikan struktur datanya benar seperti yang ditunjukkan pada Gambar 19.

```
import pandas as pd
df2 = pd.read_csv("/content/drive/MyDrive/Colab Notebooks/Machine Learning/Praktikum10/Data/weather_classification_data.csv")
df2.head()
```

	Temperature	Humidity	Wind Speed	Precipitation (%)	Cloud Cover	Atmospheric Pressure	UV Index	Season	Visibility (km)	Location	Weather Type
0	14.0	73	9.5	82.0	partly cloudy	1010.82	2	Winter	3.5	inland	Rainy
1	39.0	96	8.5	71.0	partly cloudy	1011.43	7	Spring	10.0	inland	Cloudy
2	30.0	64	7.0	16.0	clear	1018.72	5	Spring	5.5	mountain	Sunny
3	38.0	83	1.5	82.0	clear	1026.25	7	Spring	1.0	coastal	Sunny
4	27.0	74	17.0	66.0	overcast	990.67	1	Winter	2.5	mountain	Rainy

Gambar 19. Pada bagian ini ditunjukkan untuk menampilkan dataset.

```
df2.info()
```

Perintah `df2.info()` membantu untuk mengetahui tipe data setiap kolom serta memeriksa apakah terdapat nilai hilang seperti yang ditunjukkan pada Gambar 20.

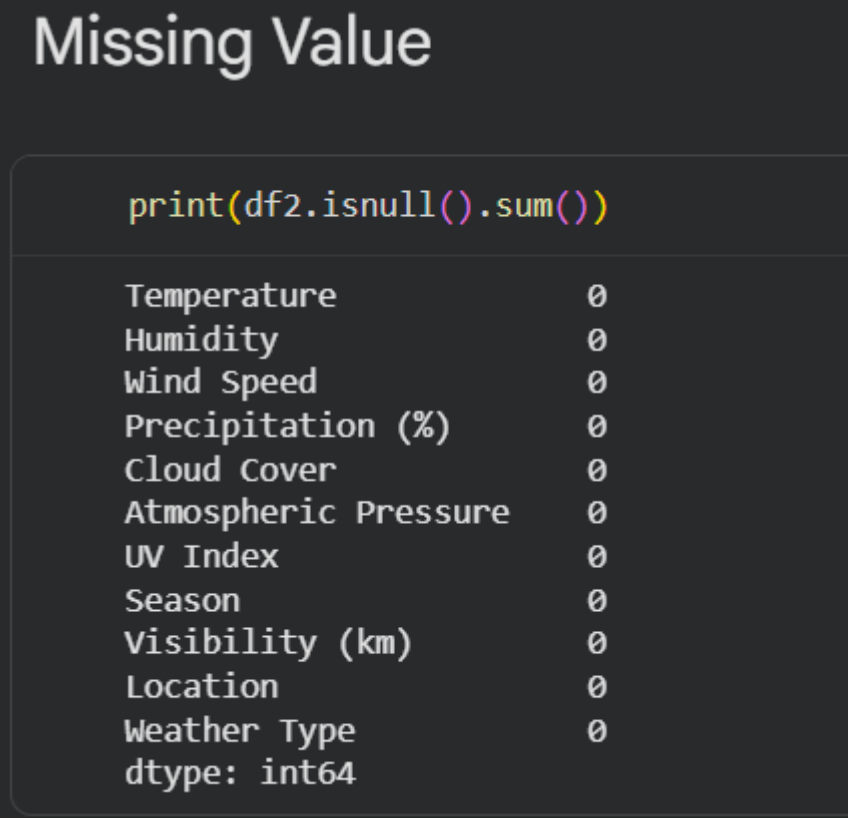
```
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13200 entries, 0 to 13199
Data columns (total 11 columns):
 #   Column                      Non-Null Count  Dtype
---  -
 0   Temperature                 13200 non-null  float64
 1   Humidity                    13200 non-null  int64
 2   Wind Speed                  13200 non-null  float64
 3   Precipitation (%)           13200 non-null  float64
 4   Cloud Cover                  13200 non-null  object
 5   Atmospheric Pressure         13200 non-null  float64
 6   UV Index                    13200 non-null  int64
 7   Season                       13200 non-null  object
 8   Visibility (km)              13200 non-null  float64
 9   Location                     13200 non-null  object
10   Weather Type                 13200 non-null  object
dtypes: float64(5), int64(2), object(4)
memory usage: 1.1+ MB
```


Gambar 20. Pada bagian ini ditunjukkan untuk informasi dataset.

```
print(df2.isnull().sum())
```

Setelah dataset tampil, dilakukan pengecekan missing value dengan `df2.isnull().sum()` dan bisa dilihat apakah terdapat nilai kosong yang harus ditangani atau tidak seperti yang ditunjukkan pada Gambar 21.



```
Missing Value
```

```
print(df2.isnull().sum())
```

Temperature	0
Humidity	0
Wind Speed	0
Precipitation (%)	0
Cloud Cover	0
Atmospheric Pressure	0
UV Index	0
Season	0
Visibility (km)	0
Location	0
Weather Type	0
dtype: int64	

Gambar 21. Pada bagian ini ditunjukkan untuk missing value.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
for col in df2.columns:
    if df2[col].dtype == 'object':
        df2[col] = le.fit_transform(df2[col])

df2.head()
```

Karena dataset ini memiliki kolom bertipe objek (teks), maka perlu dilakukan proses encoding agar dapat diubah menjadi angka. Proses tersebut dilakukan menggunakan `LabelEncoder()`, sehingga semua data kategori bisa diproses pada perhitungan jarak KNN seperti yang ditunjukkan pada Gambar 22.

Encoding Kolom Kategori

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()
for col in df2.columns:
    if df2[col].dtype == 'object':
        df2[col] = le.fit_transform(df2[col])

df2.head()
```

	Temperature	Humidity	Wind Speed	Precipitation (%)	Cloud Cover	Atmospheric Pressure	UV Index	Season	Visibility (km)	Location	Weather Type
0	14.0	73	9.5	82.0	3	1010.82	2	3	3.5	1	1
1	39.0	96	8.5	71.0	3	1011.43	7	1	10.0	1	0
2	30.0	64	7.0	16.0	0	1018.72	5	1	5.5	2	3
3	38.0	83	1.5	82.0	0	1026.25	7	1	1.0	0	3
4	27.0	74	17.0	66.0	2	990.67	1	3	2.5	2	1

Gambar 22. Pada bagian ini ditunjukkan encoding kolom kategori.

```
df2.info()
```

Lalu melakukan perintah `info()` untuk mengetahui tipe data setiap kolom serta memeriksa apakah ada nilai hilang seperti yang ditunjukkan pada Gambar 23.

```
df2.info()
```

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 13200 entries, 0 to 13199
Data columns (total 11 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Temperature                          13200 non-null  float64
 1   Humidity                            13200 non-null  int64
 2   Wind Speed                          13200 non-null  float64
 3   Precipitation (%)                   13200 non-null  float64
 4   Cloud Cover                         13200 non-null  int64
 5   Atmospheric Pressure                13200 non-null  float64
 6   UV Index                           13200 non-null  int64
 7   Season                             13200 non-null  int64
 8   Visibility (km)                     13200 non-null  float64
 9   Location                           13200 non-null  int64
10  Weather Type                        13200 non-null  int64
dtypes: float64(5), int64(6)
memory usage: 1.1 MB
```

Gambar 23. Pada bagian ini ditunjukkan informasi.

```
y = df2["Weather Type"]
X = df2.drop("Weather Type", axis=1)
```

Memisahkan fitur dan target dengan x berisi semua fitur untuk memprediksi dan y adalah label yang ingin diprediksi.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Split data training dan testing dengan data dibagi menjadi 80% training dan 20% testing, supaya model bisa belajar dan diuji kinerjanya.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Data kemudian distandarisasi menggunakan StandardScaler() agar memiliki skala yang seragam dan tidak mempengaruhi jarak antar titik yang digunakan KNN seperti yang ditunjukkan pada Gambar 24.

Memisahkan fitur X dan y

```
y = df2["Weather Type"]
X = df2.drop("Weather Type", axis=1)
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Gambar 24. Pada bagian ini menentukan fitur dan target.

```

from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Akurasi:", accuracy_score(y_test, y_pred))

```

Pada bagian pemodelan digunakan `KNeighborsClassifier(n_neighbors=5)` yang berarti model menggunakan 5 tetangga terdekat dalam menentukan hasil klasifikasi. Model lalu dilatih menggunakan data train dan kemudian dilakukan prediksi menggunakan data test dengan memanggil perintah `model.predict(X_test)` seperti yang ditunjukkan pada Gambar 25.

```

from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score

print("Akurasi:", accuracy_score(y_test, y_pred))

Akurasi: 0.8916666666666667

```

The screenshot shows a Jupyter Notebook interface. The first code cell contains the import statement for `KNeighborsClassifier`, the model initialization with `n_neighbors=5`, training the model with `model.fit(X_train, y_train)`, and making predictions with `y_pred = model.predict(X_test)`. The second code cell imports `accuracy_score` from `sklearn.metrics` and prints the accuracy score. The output of the second cell is `Akurasi: 0.8916666666666667`. A tooltip for `KNeighborsClassifier` is visible, showing the class name and a help icon.

Gambar 25. Pada bagian ini ditunjukkan prediksi.

```

from sklearn.metrics import accuracy_score, precision_score,
recall_score

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average='macro')
rec = recall_score(y_test, y_pred, average='macro')

```

```
print("Akurasi :", round(acc*100, 2), "%")
print("Precision :", round(prec*100, 2), "%")
print("Recall :", round(rec*100, 2), "%")
```

Nilai akurasi model dilihat menggunakan `accuracy_score()` dan hasilnya ditampilkan dalam bentuk persentase. Selain itu precision dan recall juga dihitung untuk menunjukkan kualitas prediksi tiap kelas secara lebih mendalam. Hasilnya dicetak dalam format persentase agar mudah dipahami seperti yang ditunjukkan pada Gambar 26.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average='macro')
rec = recall_score(y_test, y_pred, average='macro')

print("Akurasi :", round(acc*100, 2), "%")
print("Precision :", round(prec*100, 2), "%")
print("Recall :", round(rec*100, 2), "%")

Akurasi : 89.17 %
Precision : 89.22 %
Recall : 89.08 %
```

Gambar 26. Pada bagian ini ditunjukkan akurasi, precision, recall.

```
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="RdPu")
plt.title("Confusion Matrix KNN (Weather Prediction)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Untuk melihat kesalahan prediksi model pada masing-masing kelas, digunakan confusion matrix. Matriks ini divisualisasikan menggunakan heatmap warna sehingga dapat dengan mudah dilihat bagian mana model paling banyak salah prediksi seperti yang ditunjukkan pada Gambar 27.

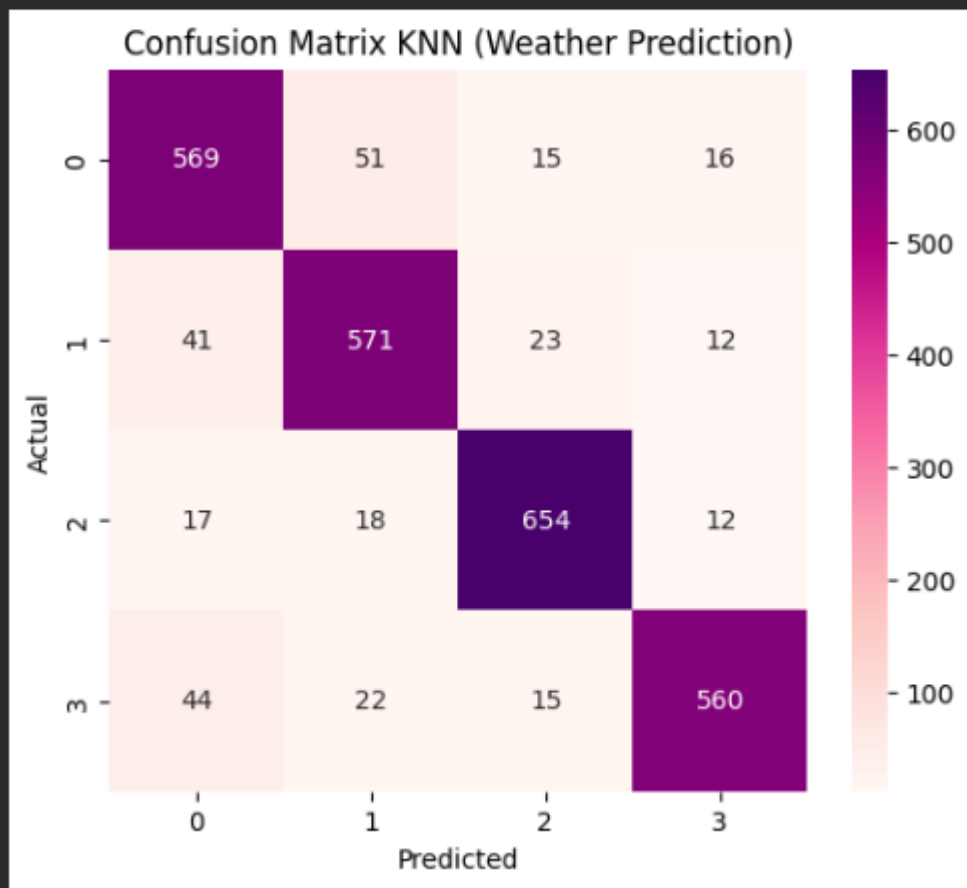
```

from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt="d", cmap="RdPu")
plt.title("Confusion Matrix KNN (Weather Prediction)")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```



Gambar 27. Pada bagian ini ditunjukkan confusion matrix.

```

from sklearn.metrics import classification_report
y_pred = model.predict(X_test)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

```

Di akhir bagian evaluasi digunakan `classification_report()` untuk menampilkan ringkasan metrik seperti precision, recall, dan f1-score tiap kategori kelas cuaca seperti yang ditunjukkan pada Gambar 28.

```
[47] ✓ Os
#perhitungan akurasi
accuracy_nb = accuracy_score(y_test, test_pred_nb)
cm_nb = confusion_matrix(y_test, test_pred_nb)
report_nb = classification_report(y_test, test_pred_nb)
print(f"\nAkurasi Naive Bayes: {accuracy_nb:4f}")

print("\nConfusion Matrix Naive Bayes:")
print(cm_nb)

print("\nClassification Report (0 = Jinak, 1 = Ganas)")
print(report_nb)
```

Akurasi Naive Bayes: 0.964912

Confusion Matrix Naive Bayes:
[[70 1]
[3 40]]

Classification Report (0 = Jinak, 1 = Ganas)

	precision	recall	f1-score	support
0	0.96	0.99	0.97	71
1	0.98	0.93	0.95	43
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

Gambar 28. Pada bagian ini ditunjukkan classification report.

```
# Menentukan nilai K terbaik menggunakan Elbow Method
error_rate = []

for k in range(1, 21): # coba K dari 1 sampai 20
    model_k = KNeighborsClassifier(n_neighbors=k)
    model_k.fit(X_train, y_train)
    pred_k = model_k.predict(X_test)
    error_rate.append(np.mean(pred_k != y_test))

# Visualisasi Elbow Method
plt.figure(figsize=(8,5))
plt.plot(range(1, 21), error_rate, marker='o', markerfacecolor='red')
plt.title("Elbow Method untuk Menentukan Nilai K Terbaik")
plt.xlabel("Jumlah K")
plt.ylabel("Error Rate")
```

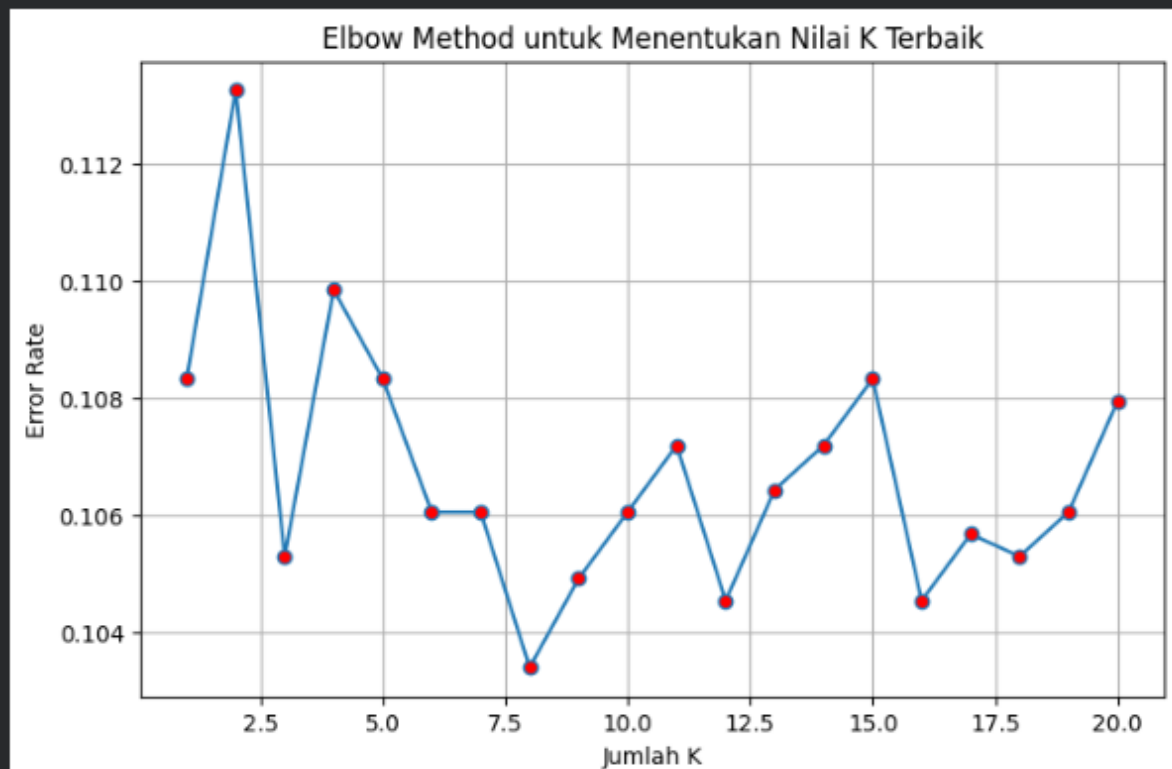
```
plt.grid(True)
plt.show()
```

Kode ini untuk Mencari titik terbaik jumlah tetangga (K). Titik siku grafik menunjukkan nilai optimal yang biasanya K sekitar 3–6. Outputnya yaitu Grafik Elbow Method seperti yang ditunjukkan pada Gambar 29.

```
# Menentukan nilai K terbaik menggunakan Elbow Method
error_rate = []

for k in range(1, 21): # coba K dari 1 sampai 20
    model_k = KNeighborsClassifier(n_neighbors=k)
    model_k.fit(X_train, y_train)
    pred_k = model_k.predict(X_test)
    error_rate.append(np.mean(pred_k != y_test))

# Visualisasi Elbow Method
plt.figure(figsize=(8,5))
plt.plot(range(1, 21), error_rate, marker='o', markerfacecolor='red')
plt.title("Elbow Method untuk Menentukan Nilai K Terbaik")
plt.xlabel("Jumlah K")
plt.ylabel("Error Rate")
plt.grid(True)
plt.show()
```



Gambar 29. Pada bagian ini ditunjukkan Elbow Method.

3. Kesimpulan dan Hasil Implementasi

Dari hasil implementasi model KNN dapat memprediksi kondisi cuaca dengan tingkat akurasi yang cukup tinggi. Ini membuktikan bahwa pola cuaca dapat dilihat berdasarkan variabel seperti temperatur, tekanan udara, dan lokasi. Model ini dapat digunakan untuk membantu sistem prakiraan cuaca berbasis data dengan waktu komputasi yang cepat dan langkah analisis yang sederhana.

Dari praktikum ini dapat disimpulkan bahwa model K-Nearest Neighbors berhasil diterapkan untuk klasifikasi cuaca pada dataset ini. Dengan pemilihan jumlah tetangga terbaik menggunakan Elbow Method, model dapat mencapai akurasi yang baik. Semua fitur memainkan peran penting dalam memprediksi *Weather Type* sehingga KNN layak digunakan sebagai solusi prediksi cuaca berbasis data. Proses encoding, scaling, dan evaluasi memberi pemahaman lebih jelas mengenai bagaimana machine learning bekerja dalam mempelajari pola data untuk mengambil keputusan yang tepat.

KESIMPULAN

Berdasarkan seluruh percobaan yang telah dilakukan pada ketiga latihan, dapat disimpulkan bahwa algoritma K-Nearest Neighbors merupakan metode klasifikasi yang mampu memberikan hasil yang kompetitif untuk berbagai tipe data. Proses preprocessing seperti encoding data kategorikal dan normalisasi sangat mempengaruhi kualitas model sehingga tahap tersebut wajib dilakukan sebelum pelatihan model. Pemilihan nilai K terbaik melalui metode seperti Elbow Method juga berperan penting dalam meningkatkan akurasi prediksi. Pada latihan pertama dan ketiga, KNN mampu melakukan prediksi dengan baik sehingga dapat membantu proses pengambilan keputusan seperti menentukan kondisi cuaca atau kenyamanan seseorang terhadap keadaan lingkungan.

Sementara itu, dari hasil evaluasi pada latihan kedua terlihat bahwa model klasifikasi tidak hanya dinilai dari akurasi, tetapi juga perlu melihat nilai precision dan recall untuk memahami lebih dalam performa sistem dalam menangani kasus benar dan salah klasifikasi. Evaluasi menggunakan confusion matrix membantu mengidentifikasi pola kesalahan prediksi sehingga dapat dijadikan dasar perbaikan model lebih lanjut. Melalui seluruh rangkaian praktikum ini dapat disimpulkan bahwa penerapan machine learning, khususnya algoritma KNN, memiliki potensi yang sangat berguna dalam berbagai konteks penerapan nyata, namun tetap diperlukan pemahaman mendalam dalam pemilihan parameter dan evaluasi model untuk mendapatkan hasil yang optimal.

Link GitHub (Praktikum dan Latihan Pertemuan 10)

https://github.com/revani18/ML_2025_Revani_3AI01/tree/795d639396b53f008823a32f5f7b4ae9d4dba0d/Praktikum10

Praktikum:

https://github.com/revani18/ML_2025_Revani_3AI01/blob/795d639396b53f008823a32f5f7b4ae9d4dba0d/Praktikum10/Notebooks/praktikum10.ipynb

Latihan:

https://github.com/revani18/ML_2025_Revani_3AI01/blob/795d639396b53f008823a32f5f7b4ae9d4dba0d/Praktikum10/Notebooks/latihan.ipynb